

Arduino 演奏システム設計書

25G1051 近藤巧望

2026 年 5 月 13 日

1 担当箇所

本設計書は，Arduino を用いた演奏システムのトロンボーンユニット及び，演奏同期機能の設計に関するものである．目標としては，同期における各楽器ユニット間との遅延を最小化し，リアルタイムな演奏制御を実現することにある．

2 システムの基本設計

2.1 機能一覧

トロンボーンユニットの主要機能とその詳細を表 1 にまとめる．実装にあたっては，これらの機能が相互に連携し，リアルタイムな演奏制御を実現することが求められる．

Processing において，トロンボーンの音色は振幅で制御するため F2.1 の振幅ベース音色制御が重要な役割を果たす．音響物理学・音色合成に関する理論では，加算合成法に基づき，基本周波数とその倍音成分の振幅比率を調整することで特定の楽器音色を模倣できることが知られている [1]．

表 1 トロンボーンシステム機能一覧表

ID	主要機能	機能詳細・役割	対応要素
F1.0	データ同期制御	外部入力値を内部演奏パラメータへ動的にマッピングする。	-
F1.1	パラメータ同期受信	Arduino からの BPM・音名データを変数へ即時反映し同期させる。	currentBpm, currentNote
F1.2	演奏時間動的計算	受信した BPM 値に基づき、音価に対応した実時間を算出する。	beatLength, noteLength
F2.0	音響信号生成系	パラメータ制御による波形合成および信号処理を行う。	-
F2.1	振幅ベース音色制御	各倍音の振幅値（比率）を直接指定し、トロンボーン音色を合成する。	wave1 ~ wave4, amp
F2.2	音階・周波数変換	受信した音名文字列を、DSP 処理用の周波数データへ変換する。	noteToFreq()
F2.3	信号エンベロープ処理	指定された振幅最大値（maxAmp）に基づき、時間的減衰を制御する。	Line (env1 ~ env4)
F3.0	出力・表示	処理結果を視聴覚情報としてユーザへ提示する。	-
F3.1	内部状態表示	BPM, Note 名, 振幅設定値などの稼働パラメータを文字表示する。	draw()

2.2 システム構成図

トロンボーンユニットのシステム構成を図 1 に示す。UI/制御系は PC 上のブラウザで実装され、Arduino サーバーと WiFi (HTTP/OSC) を介して通信する。Arduino サーバーは、受信した BPM 情報をもとに全体の演奏テンポを管理し、トロンボーンユニットを含む各楽器ユニットへ同期信号（パルス信号）を小節ごとに送信する。トロンボーンユニットは、受信した同期信号に基づいて自身の演奏タイミングを確定させ、Processing へ演奏開始のトリガーを送信する。Processing は、受信したトリガーに基づいて音響信号（振幅制御による合成音声）を生成し、PC 内蔵スピーカーから出力する。

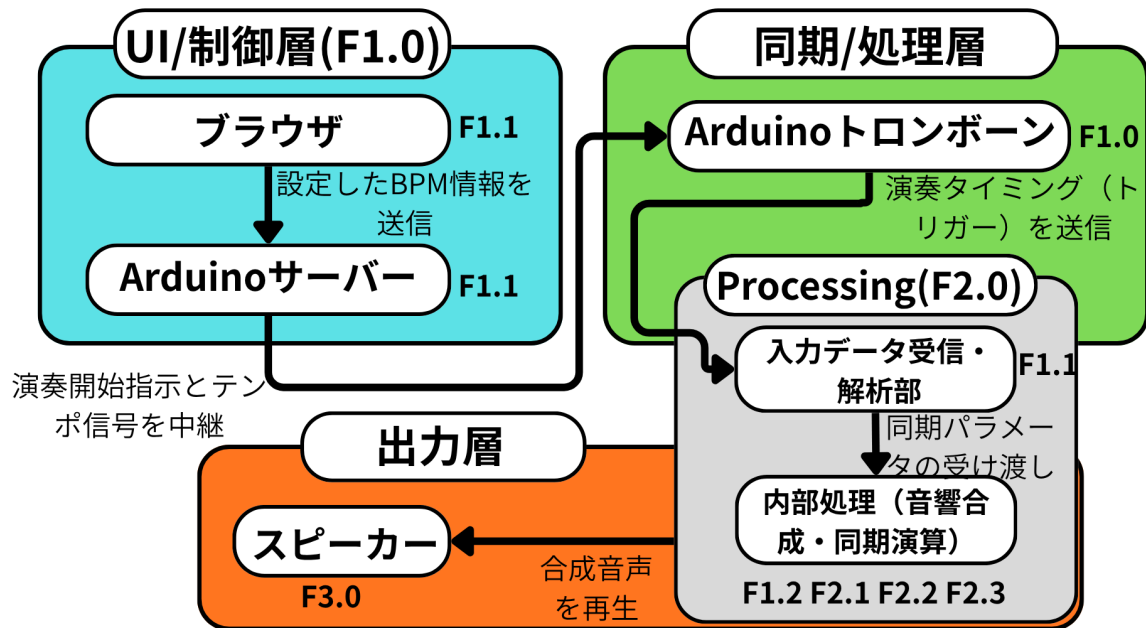


図1 トロンボーン部分のシステム構成図、表1の機能IDを対応させて記載している。

2.3 操作インターフェース設計

ユーザは、PC上のブラウザに実装されたWeb UIを通じてシステムを操作する。本インターフェースは以下の機能を備える。

- **BPM 入力フィールド**：楽曲のテンポを指定し、Arduino サーバーへ WiFi (HTTP/OSC) 経由で送信する。
- **接続ステータス表示**：各 Arduino ユニットとの通信確立状況をリアルタイムで確認できる。
- **演奏制御ボタン**：シーケンスの開始 (送信) および強制停止を命令する。

2.4 状態遷移図

本ユニット (Arduino トロンボーン) は、外部機器との同期を自動的に行うため、図2に示す4つの状態を遷移する。各状態は機能一覧の各項目と密接に関連している。

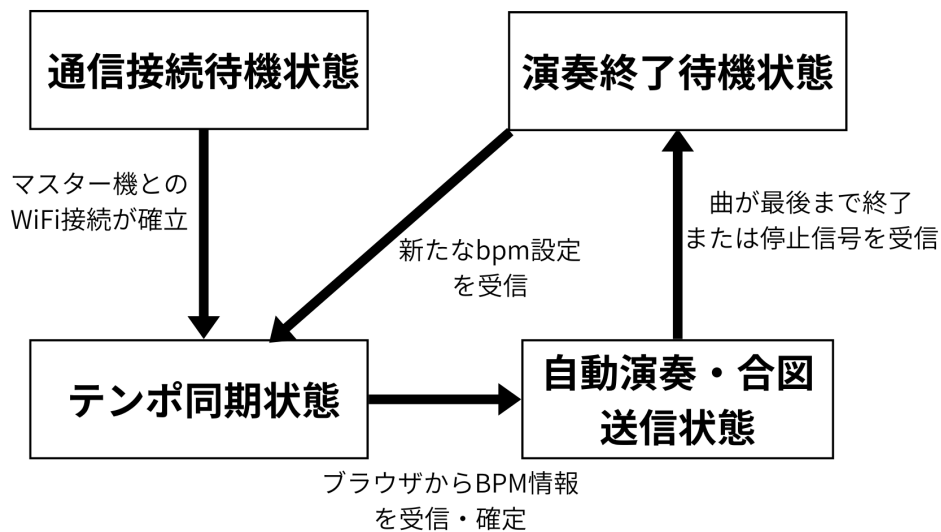


図2 トロンボーンユニットの状態遷移図

各状態の役割および遷移条件の詳細を以下に示す。

- **1. 通信接続待機状態**：電源投入後の初期状態である。Arduino サーバーとの WiFi 接続を確立し、データの受信準備を整える。接続の確立が次状態へのトリガーとなる。
- **2. テンポ同期状態 (F1.1, F1.2)**：ブラウザ UI から確定された BPM 情報を受信し、内部変数 (currentBpm) を更新する状態である。受信した数値に基づき、音価 (音符の長さ) に対応する実時間を計算する。
- **3. 自動演奏・合図送信状態 (F2.0 全般)**：演奏開始信号を受信した瞬間に移行する実行状態である。プログラムされた音階データを処理しつつ、第 1 拍目のタイミングで Processing へ同期トリガーを送信する。この間、振幅制御に基づいた音声合成が並行して実行される。
- **4. 演奏終了待機状態**：楽曲の終了、または手動の停止信号を受信した際の安全状態である。すべての音声出力を停止し、変数をリセットする。新たな BPM 設定を受信することで、再び「テンポ同期状態」へと復帰する。

3 システムの詳細設計

3.1 部品・ソフトウェア一覧

本システムの構築に使用するハードウェアおよびソフトウェア一式を表 2 に示す。まず、前提として OS は macOS を使用する。また、Arduino Uno R4 は WiFi 通信機能を内蔵しているため、外部モジュールは必要ない。スピーカーに関しては PC 内蔵のものを使用するため、特別なオーディオインターフェースも不要である。音声を作成するための Processing では、Minim ライブラリを使用する。Minim は Processing で音響処理をするためのライブラリであり、波形の合成やエンベロープ制御などの機能を提供する [2]。更に、

Arduino にプログラム書き込むためには Arduino IDE を使用する。通信プロトコルは、UDP をベースとした OSC (Open Sound Control) を採用する。OSC は音楽やマルチメディアアプリケーション間の通信に特化したプロトコルであり、低遅延かつ柔軟なデータ構造を持つため、本システムのリアルタイムな演奏同期に適している [3]。さらに、演奏のエンタメ性を向上させる視覚的演出用のアクチュエータとしてサーボモータを使用する。また、システム全体のテンポをリアルタイムに調整するため、BPM 操作用のタクトスイッチを 2 基搭載する。

表 2 部品・ソフトウェア一覧

分類	名称・型番	用途・備考
ハードウェア	PC	Processing の実行，ブラウザ UI のホスト。
	Arduino Uno R4 WiFi	サーバー機およびトロンボーン機として計 2 台使用 (他楽器を含めると全部で 5 台使用)。WiFi 通信機能必須。
	サーボモータ (SG90)	演奏に連動した視覚的演出 (動作) の実行。
	タクトスイッチ	BPM の増減操作作用として各ユニットに 2 基ずつ使用。
ソフトウェア	Processing	音響合成エンジンおよびメインシーケンサー。
	Minim ライブラリ	Processing 上での波形合成，エンベロープ制御。
	Arduino IDE	マイコン用制御プログラムの開発。
通信プロトコル	UDP / OSC (Open Sound Control)	各ユニット間での低遅延な演奏同期信号の送受信。

3.2 ハードウェア設計

本項では，前節の表 2 に基づき，システム全体を統括するサーバー機の具体的な接続関係および動作仕様について述べる。

3.2.1 周辺部品の詳細仕様

サーバー機は，システム全体の演奏テンポを管理し，各楽器ユニットへ同期信号を配信する役割を担う。そのため，物理インターフェースとして BPM 調整用のタクトスイッチを 2 基，および演奏に連動した視覚演出を行うためのサーボモータを搭載する。

表 3 使用部品の詳細仕様（サーバー機）

名称	型番・メーカー	端子/ピン	特性・役割
BPM UP ボタン	タクトスイッチ	D2	BPM を増加（例：+5）させるための物理入力。内部プルアップを使用。
BPM DOWN ボタン	タクトスイッチ	D3	BPM を減少（例：-5）させるための物理入力。内部プルアップを使用。
演出用モータ	サーボモータ (SG90)	D9	演奏テンポや同期信号に連動し、視覚的演出を行うためのアクチュエータ。
給電用ケーブル	USB Type-C ケーブル	電源入力	Arduino への給電およびデバッグ用シリアル通信に使用。

3.2.2 回路構成と接続インターフェース

Arduino Uno R4 WiFi を中心とした、サーバー機の接続構成を図 3 に示す。

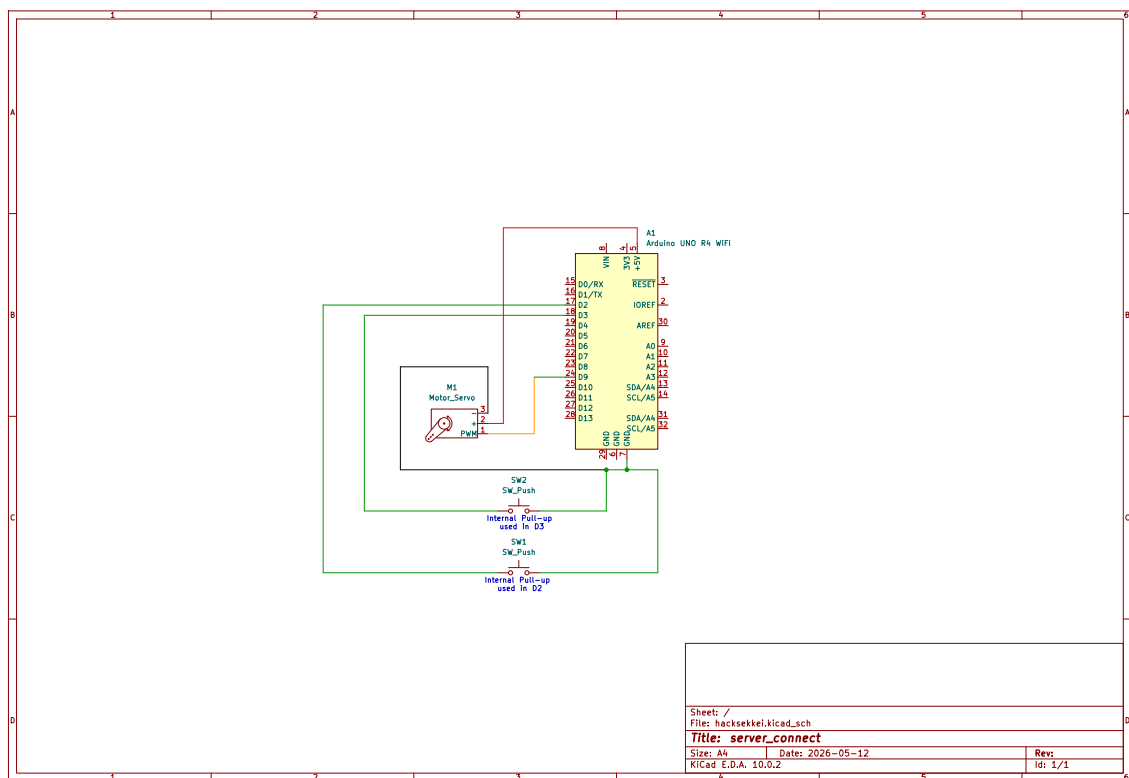


図 3 サーバー機の回路構成図。

具体的な回路接続として、BPM 操作用の各タクトスイッチは、デジタル入力ピン D2 および D3 と GND の間にそれぞれ接続される。Arduino 内部のプルアップ抵抗（INPUT_PULLUP）を有効化し、押下時に LOW を検知するアクティブラー構成を採用することで、外付け抵抗を排した堅牢な回路としている [4]。これは、ユー

ザが BPM を直感的に変更できるようにするための物理ボタンであり、難しい操作なしにテンポの調整が可能となる。プルアップ抵抗を使用する理由としては、スイッチ解放時の入力ピンが不安定な状態になるのを防ぎ、ノイズによる誤作動を除去するためである。

また、演出用のアクチュエータであるサーボモータは、制御信号線を PWM 出力に対応した D9 ピンに接続し、電源供給は Arduino の 5V および GND ピンより行う構成とする。

3.2.3 具体的なハードウェア動作

各部品および通信機能が連携し、演奏同期を実現するプロセスを以下に定義する。

1. **ネットワーク同期パケットの配信**：サーバー機は、内部タイマーに基づき「小節番号パケット」を WiFi (UDP/OSC) 経由で全楽器ユニットへ配信する。各ユニットはこのパケットを受信した瞬間に自身の担当パートの演奏を開始する。
2. **物理ボタンによる BPM 更新と反映**：D2 および D3 ピンの入力を検知した際、プログラム内の BPM 変数を即座に更新する。これにより、次に配信される小節番号パケットの送信間隔が動的に変化し、システム全体の演奏テンポが同期して変更される。
3. **演出用モータの連動制御**：設定された BPM および配信タイミングに連動し、D9 ピンに接続されたサーボモータを駆動させる。これにより、メトロノームのような視覚的なテンポガイド、あるいは演奏に合わせたエンタメ性の高い物理動作を実現する。
4. **通信モジュールの最適化**：Arduino Uno R4 WiFi の内蔵モジュールを用いることで、外部配線によるノイズや断線リスクを排除し、パケット送信の遅延を最小化している。

3.3 ソフトウェア設計

3.3.1 ファイル構成

トロンボーンユニットの制御プログラムは、保守性や他ユニットとの混同を避けるため、以下のモジュール構成をとる。

- `Trombone_Unit.ino`：メインプログラム。初期化処理および受信イベントの常時監視を行う。
- `OscHandler.cpp` / `.h`：サーバーから受信した OSC パケット（小節番号・BPM 情報）のパーズ処理。
- `SynthTrigger.cpp` / `.h`：受信した同期信号を Processing が解釈可能な形式に変換し、転送するロジック。
- `Secret.h`：WiFi の SSID、パスワード、およびサーバー・PC の固定 IP アドレスを定義する。

3.3.2 オブジェクト設計（クラス図）

各機能をカプセル化し、リアルタイム性を確保するためのクラス設計を行う。

- **OscReceiver クラス**：特定の UDP ポートをリッスンし、サーバー機からパケットが到達した際にコールバック関数を呼び出す。
- **SequenceController クラス**：現在の小節番号を保持し、自身の担当パートであるか判定する。また、受信した BPM 値を保持し、演出用サーボの動作速度を補正する。

- **ActionManager** クラス：演奏トリガーを Processing へ射出すると同時に、サーボモータの物理動作を実行する。

3.3.3 関数設計

各クラスに実装される主要な関数の役割を以下に定義する。

- `void handleOscMessage()`：受信パケットの OSC アドレスを確認し、アドレスが `/sync/measure` であれば小節番号を、`/sync/bpm` であればテンポ値を抽出して各クラスへ振り分ける。
- `void triggerNote(int measure)`：引数の小節番号に基づき、自身の発音タイミングであれば Processing へ OSC メッセージ `/play/trigger` を送信する。
- `void moveVisualServo(float bpm)`：BPM に基づき、サーボモータの動作角度とスピードを計算し、視覚的な演奏演出を実行する。

3.3.4 主要関数の実装例と動作仕様

設計した各関数がどのようにシステム内で機能するか、以下に具体的なコード例を示す。`handleOscMessage()` は、サーバーから届いた OSC パケットのアドレスを読み取り、適切な変数へ値を格納する処理である。使用例は以下の通りである。

Listing 1 `handleOscMessage` の実装例

```
void handleOscMessage(OSCMessage &msg) {
// 小節番号 (/sync/) の受信処理measure
if (msg.fullMatch("/sync/measure")) {
int receivedMeasure = msg.getInt(0);
triggerNote(receivedMeasure); // 発音判定関数へ
}
// 情報 (BPM/sync/) の受信処理bpm
else if (msg.fullMatch("/sync/bpm")) {
currentBpm = msg.getFloat(0);
// サーボの動作速度をに合わせて更新BPM
}
}
```

`triggerNote(int measure)` は、受信した小節番号があらかじめ割り当てられたトロンボーンの担当パートと一致するかを判定し、Processing へ発音指示を送る。使用例は以下の通りである。

Listing 2 `triggerNote` の実装例

```
void triggerNote(int measure) {
// 自身の担当小節 (例：小節, 小節) かチェック 48...
if (measure == myTargetMeasure) {
OSCMessage msg("/play/trigger");
```



```

msg.add("trombone"); // 楽器名を指定
Udp.beginPacket(pcIp, pcPort);
msg.send(Udp);
Udp.endPacket();

```

```

// 演奏に合わせてサーボを動かす
moveVisualServo(currentBpm);
}
}

```

moveVisualServo(float bpm) は、BPM に基づいてサーボモータの動きを制御し、BPM が高いほどサーボの動作速度を速める計算式を導入する。使用例は以下の通りである。

Listing 3 moveVisualServo の実装例

```

void moveVisualServo(float bpm) {
// から拍の時間をミリ秒で計算BPM1
int beatDuration = 60000 / bpm;

// トロンボーンのスライドを模した動作（度から度へ）090
myServo.write(90);
delay(beatDuration / 4); // 分音符分だけ待機して戻す16
myServo.write(0);
}

```

3.4 処理フローの設計

本システム（トロンボーンユニット）は、サーバー機からの同期信号を基軸とした以下の3段階のフローによって動作する。

1. **初期化**：起動時に WiFi 接続を確立し、サーバー機から配信される OSC パケット（小節番号・BPM）の受信待機状態へ移行する。
2. **同期・解析**：受信した OSC パケットを解析し、現在の全体のテンポおよび現在の小節番号を内部変数に反映する。
3. **実行および演出**：受信した小節番号が自身の担当パートである場合、Processing へ演奏トリガーを送信する。同時に、BPM に基づいた速度計算を行い、サーボモータによる視覚的演出を実行する。

また、図4にトロンボーンユニットにおける処理フローの概要を示す。

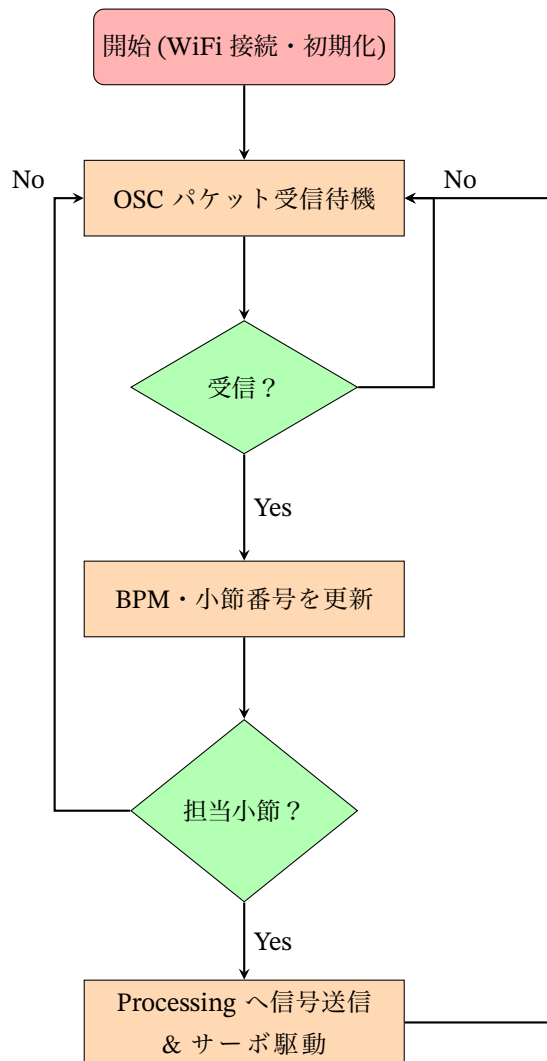


図4 トロンボーンユニットの処理フロー

プログラム開始直後の `setup()` 関数は初期化フェーズに該当する。WiFi への接続および OSC ポートの開放を行い、サーバーからのパケットを受け取れる状態を構築する。また、サーボモータを初期位置へリセットする処理もこの段階で実行される。

メインループ内で常時呼び出される `handleOscMessage()` 関数は、パケットの受信と解析を担当する。サーバーから配信されたパケットが BPM に関するものか小節番号に関するものを識別し、内部の変数へ最新の値を反映させる役割を持つ。

小節番号を受信した際に `triggerNote()` 関数内で行われる論理チェックは、担当パートの判定フェーズにあたる。全ユニットに一斉配信される同期信号の中から、自身の担当する小節番号との一致を確認することで、各楽器が適切なタイミングで演奏を開始する自律的な制御を実現している。

判定が一致した際に実行される `triggerNote()` および `moveVisualServo()` は、演奏と演出の実行プロセスを担う。Processing に対して音響合成のトリガーを送出すると同時に、現在の BPM 値から計算された速度に基づき、サーボモータがスライドの動きなどの物理的な演奏動作を再現する。

3.5 テストの設計

サーバー機から配信される同期信号に対し、トロンボーンユニットが正確かつ遅延なく追従するかを検証するため動作テストを実施する。

BPM 追従性テストでは、サーバー機側で BPM を変更した際、トロンボーンユニットが受信したパケットから即座にテンポを解析し、内部の演奏周期およびサーボモータの動作速度に正しく反映されているかを確認する。

発音同期タイミングテストでは、トロンボーンユニットが特定の小節番号を受信した瞬間に射出する演奏トリガーと、Processing 側で生成される音響および伴奏の開始タイミングを比較する。これにより、ネットワーク経由での通信遅延が演奏に支障をきたさない許容範囲内であることを検証する。

視覚演出連動テストでは、Processing から出力される楽器音のエンベロープ（音の立ち上がりと減衰）に対し、サーボモータによる物理的な動作が視覚的に違和感なく同期しているかを確認する。

ネットワーク安定性テストでは、複数の楽器ユニットが同時に接続された環境下において、トロンボーンユニットが混信やパケットロスの影響を受けず、サーバーからのパケットを正確にフィルタリングして自身の担当パートのみを確実に実行できるかを検証する。

参考文献

- [1] N. H. Fletcher and T. D. Rossing (2012). The physics of musical instruments. https://books.google.com/books?hl=ja&lr=lang_ja/lang_en&id=gVDSBwAAQBAJ&oi=fnd&pg=PR5&dq=N.+H.+Fletcher+%26+T.+D.+Rossing+%22The+Physics+of+Musical+Instruments%22&ots=noY7enAyw9&sig=St1TtAdA4Qiw9_d_x1au4-K37Mw
- [2] Minim document. <https://code.compartmental.net/minim/>
- [3] Wright, M., & Freed, A. (1997). Open Sound Control: A New Protocol for Communicating with Sound Synthesizers. Proceedings of the International Computer Music. <https://www.adrianfreed.com/sites/default/files/open-soundcontrol-a-new-protocol-for-communicating-with.pdf>
- [4] Arduino Docs. "INPUT | INPUT_PULLUP | OUTPUT" <https://docs.arduino.cc/language-reference/en/variables/constants/inputOutputPullup/>